VASTRA

PHASE 1 — THREAT MODEL

Design Review Submission | 19 October 2026

Amay Sharma UID: 25LBCS3018	Kumar Shailendra UID: 25LBCS3293

1. Purpose and System Scope

Vastra is a proposed clothing discovery and comparison web application. Users search for apparel across supported retailer sources, compare available product details, and may save products to a wishlist. Customer reviews are displayed only when a supported source returns identifiable review records; the system must not fabricate reviews. The proposed architecture is a React frontend, a Node.js/Express REST API, Azure SQL Database, and optional Azure Blob Storage.

2. Assets to Protect

Asset	Security / privacy concern
User accounts	Email, password hash, role, and account metadata must be protected.
Wishlist and search history	Prevent access by other users; collect and retain only what is needed.
Product and review data	Preserve source links and provenance; treat external content as untrusted.
Secrets and database credentials	Prevent exposure in source code, repository history, and logs.
Activity and source-health logs	Restrict access and avoid recording passwords, tokens, or unnecessary personal data.

3. Main Threats and Mitigations

Threat	Possible impact	Planned mitigation
Unauthorized access / broken access control	A user views another user's wishlist or accesses administrator functions.	Require authentication; enforce ownership checks and role-based authorization on the server; test user/admin permissions.
Password exposure	Account compromise if credentials are disclosed.	Store only password hashes using a modern password-hashing algorithm; never log passwords; use secure authentication flows.
SQL injection	Unauthorized database reads or changes.	Use parameterized queries or safe ORM methods; validate types and lengths; allow-list sort fields; apply least-privilege DB access.
Cross-site scripting (XSS)	Untrusted product or review text executes in a user's browser.	Escape/encode untrusted output; do not inject raw HTML; validate and normalise external fields.
Secret leakage	Attackers gain access to APIs or cloud resources.	Keep secrets in environment configuration or a managed secret store; commit placeholders only; review repository history.
Database network exposure	Direct access to the relational database from untrusted networks.	Configure Azure SQL firewall/network rules; allow only required connections; use least-privilege database credentials.
Transport interception	Credentials or session data are exposed in transit.	Use HTTPS in deployment; configure secure cookie/token settings; avoid sensitive data in URLs.
Malicious or unreliable retailer data	Malformed data, misleading content, or stored XSS reaches users.	Treat source responses as untrusted; validate/normalise data; retain source URLs and retrieval timestamps where practical.
Abuse or external-source outage	Excessive requests, high costs, or searches that hang when a source fails.	Apply rate limits, bounded concurrency and timeouts; handle connector failures safely; return partial results where possible.
Sensitive activity logs	Logs reveal credentials, tokens, or unnecessary personal data.	Log security events and source failures only as needed; restrict log access; exclude passwords, tokens, and sensitive values.

4. Trust Boundaries and Data Flow

Trust boundaries: (1) the user's browser to the hosted API; (2) the API to Azure SQL Database and optional Blob Storage; (3) the API to external retailer websites/APIs. Validate requests at the API boundary, authorize every protected action on the server, use encrypted connections, and treat all external retailer content as untrusted. Store only necessary data and use dummy or public data for the academic demonstration.

5. Security Checks Before Review

- Authentication works and passwords are hashed.
- Role-based access is enforced server-side; users cannot access another user's wishlist.
- SQL inputs are parameterized and user-supplied sort/filter values are allow-listed.
- Product and review text is safely rendered to prevent XSS.
- HTTPS is enabled for the deployed application.
- Secrets are excluded from source control and logs.
- Azure SQL firewall/network rules restrict access.
- Activity logs record relevant events without credentials or tokens.
- Source connectors have timeouts, rate limits, and safe failure handling.

6. Assumptions and Status

This is a proposed Phase 1 threat model, not evidence that controls have already been implemented or tested. The final hosting setup, retailer data sources, and Azure configuration must be confirmed during implementation. Update this document if the deployed architecture or data handling changes.